

ARCHITECTURE & ENGINEERING

Dari Analisis Log Hingga Respons Otomatis Terintegrasi

Disusun Oleh: **Rizki Angga Pratama**
L2 SOC Analyst

HAK CIPTA & DISCLAIMER

SOC Architecture & Engineering: Dari Analisis Log Hingga Respons Otomatis Terintegrasi
Edisi Enterprise (50 Halaman)

Disusun oleh: Rizki Angga Pratama, S. Kom, C. Pro

Dokumen ini merupakan kompilasi cetak biru teknis, metodologi deteksi sistem, dan arsitektur respons insiden yang dirancang khusus untuk praktisi *Security Operations Center* (SOC), Threat Hunters, dan Security Engineers.

> **Disclaimer Operasional:**
> Seluruh cuplikan kode (KQL, EQL, Python API, JSON, dan LoRA parameters) yang disajikan dalam buku ini merupakan ilustrasi teknis. Implementasi pada lingkungan *Production* (Produksi) harus melalui proses *Quality Assurance* (QA), *Shadow Mode*, dan penyesuaian (*Tuning*) terhadap arsitektur IT masing-masing organisasi. Penulis tidak bertanggung jawab atas pemadaman sistem (*downtime*) atau kegagalan deteksi akibat penyalinan kueri tanpa validasi.

Filosofi Desain:
Dirancang menggunakan estetika *Dark Mode* untuk meniru antarmuka terminal dan SIEM. Palet warna yang digunakan (Emerald, Cyan, Amber) dipilih secara spesifik untuk mengurangi kelelahan kognitif selama sesi investigasi panjang.

© 2026 Hak cipta dilindungi undang-undang. Dokumen ini dilisensikan untuk penggunaan pelatihan Cybersecurity.

KATA PENGANTAR

Evolusi pusat operasi keamanan (*Security Operations Center* / SOC) selama dekade terakhir telah mencapai titik jenuh struktural. Kita telah bergerak dari era di mana analis secara manual menatap log *firewall* yang bergulir di layar terminal, menuju era sentralisasi *Security Information and Event Management* (SIEM). Namun, dengan sentralisasi tersebut, datanglah musuh baru yang tak terlihat: **Alert Fatigue**.

Hari ini, musuh terbesar kita bukanlah grup peretas APT atau *ransomware zero-day*, melainkan beban kognitif (*cognitive load*). Saat seorang analis dihadapkan pada 5.000 peringatan merah setiap hari—di mana 99% darinya adalah *false positive* administratif—kelelahannya bukan lagi soal fisik, melainkan sistemik. Ketumpulan analitis ini menyebabkan peringatan nyata terabaikan, dan pelanggaran data yang sesungguhnya pun terjadi.

Buku *SOC Architecture & Engineering* ini lahir sebagai antitesis terhadap operasi SOC yang reaktif. Buku ini adalah manifes untuk transisi peran: dari sekadar "Pengamat Layar" (*Screen Watcher*) menjadi seorang **Detection & Response Engineer**.

Visi utama dari *master e-book* 50 halaman ini adalah menjembatani seluruh pilar pertahanan modern ke dalam satu alur kerja yang menyatu. Kita memulainya dari dasar—membedah lautan data dengan kueri elastis tingkat tinggi (KQL & EQL), membangun logika deteksi dan batas peredaman (*suppression*) yang cerdas. Kemudian, kita mengalirkan deteksi tersebut ke dalam mesin orkestrasi (SOAR) untuk memotong *Mean Time to Respond* (MTTR) menjadi hitungan sub-detik pada berbagai lanskap, mulai dari perimeter korporat hingga awan (*Cloud*).

Karya ini saya persembahkan sebagai cetak biru pertahanan di dunia nyata. Selamat datang di era *Next-Generation SecOps*.

— Rizki Angga Pratama

Author

DAFTAR ISI (1/2)

BAGIAN I: Visibilitas & Deteksi

Bab 1.1 Intro The Elastic Security Masterclass Hlm. 7

Bab 1.2 Arsitektur ELK & Pipeline Data Hlm. 8

Bab 1.3 Standarisasi ECS (Elastic Common Schema) Hlm. 9

Bab 1.4 KQL vs EQL: Stateful vs Stateless Hlm. 10

Bab 2.1 Machine Learning di Lingkungan SOC Hlm. 11

Bab 2.2 Membangun Anomaly Baseline Hlm. 12

Bab 2.3 Tuning & Poisoning ML Jobs Hlm. 13

Bab 3.1 The Detection Engineering Blueprint Hlm. 14

Bab 3.2 Whitelisting vs Suppression Hlm. 15

Bab 3.3 Logika Threshold & Agregasi Hlm. 16

Bab 3.4 Shadow Mode & Quality Assurance Hlm. 17

BAGIAN II: Orkestrasi & Respons Taktis

Bab 4.1 Arsitektur SOAR (Security Orchestration) Hlm. 19

Bab 4.2 Malware Containment Playbook Hlm. 20

Bab 4.3 Integrasi API (Python ke EDR) Hlm. 21

Bab 5.1 Perimeter Defense: Ingress vs Egress Hlm. 22

Bab 5.2 Manajemen Scanner & Noise Publik Hlm. 23

Bab 5.3 External Dynamic Lists (EDL) Firewall Hlm. 24

Bab 6.1 Cloud Incident Response (AWS/Azure) Hlm. 25

Bab 6.2 Analisis Log CloudTrail Hlm. 26

Bab 6.3 IAM Quarantine Automation Hlm. 27

DAFTAR ISI (2/2)

BAGIAN III: Proaktif & Tooling

Bab 7.1 Purple Teaming Methodology Hlm. 29

Bab 7.2 Atomic Red Team Emulation Flow Hlm. 30

Bab 7.3 Visualisasi MITRE ATT&CK Heatmap Hlm. 31

Bab 8.1 SecOps Tooling: Custom Interfaces Hlm. 32

Bab 8.2 JavaScript RegEx Parser (Client-Side) Hlm. 33

Bab 8.3 Fetching API & Mengatasi CORS Hlm. 34

Bab 9.1 Interactive SecOps Dashboards Hlm. 35

Bab 9.2 Arsitektur Bento Grid Layout Hlm. 36

Bab 9.3 Psikologi Warna & Ergonomi Kognitif Hlm. 37

BAGIAN IV: AI & Otomatisasi Lanjutan

Bab 10.1 AI-Driven SOC: Menggeser Beban Analis Hlm. 39

Bab 10.2 Arsitektur RAG (Retrieval-Augmented Gen) Hlm. 40

Bab 10.3 System Prompt Engineering untuk JSON Hlm. 41

Bab 10.4 LoRA Fine-Tuning pada LLM Lokal Hlm. 42

BAGIAN V: Studi Kasus & Referensi

Bab 11.1 Studi Kasus: Deteksi & Containment Ransomware Hlm. 44

Bab 11.2 Attack Timeline Reconstruction Hlm. 45

Bab 12.1 Studi Kasus: Insider Threat Data Exfiltration Hlm. 46

Bab 12.2 EQL Correlation Query Breakdown Hlm. 47

Glosarium Istilah SecOps (A-M) Hlm. 48

Glosarium Istilah SecOps (N-Z) Hlm. 49



BAGIAN I

Visibilitas, Analitik Log,
& Rekayasa Deteksi

BAB 1: THE ELASTIC SECURITY MASTERCLASS

1.1 Menguasai Lautan Data Log

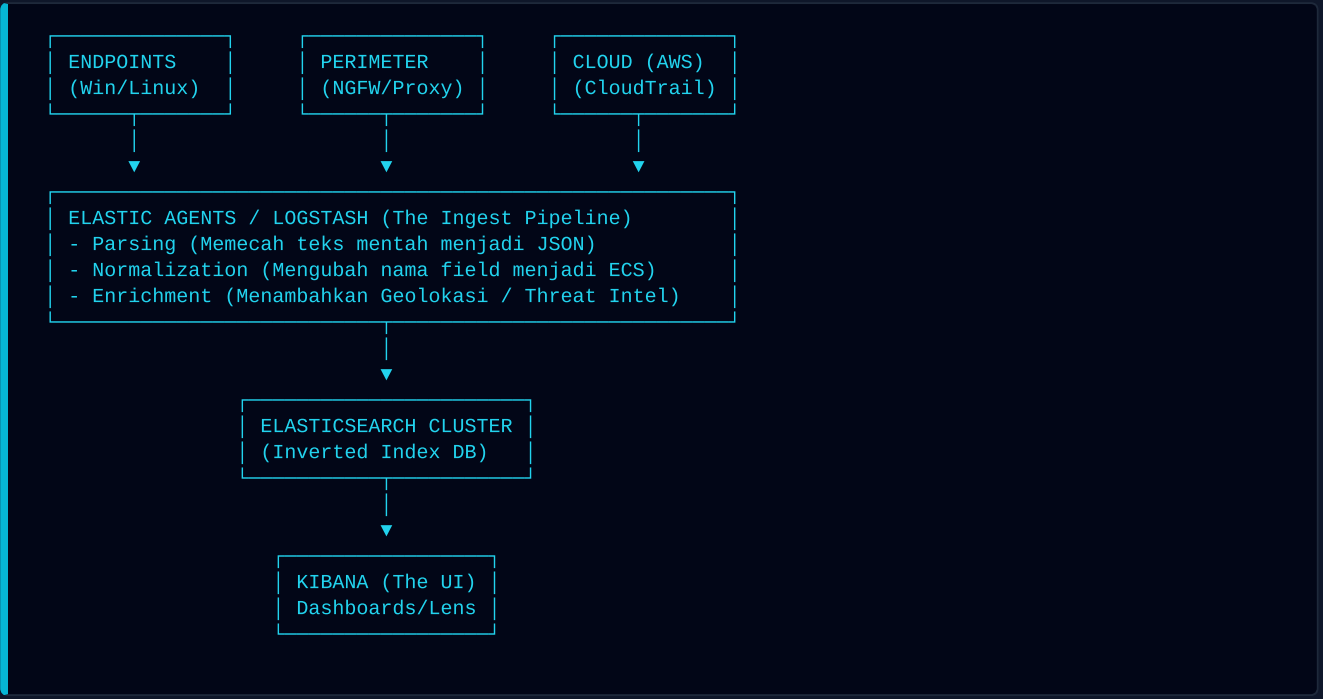
Di era keamanan modern, SOC (*Security Operations Center*) tidak lagi menderita karena kekurangan data; mereka tenggelam di dalamnya. Sebuah perusahaan skala menengah rata-rata menghasilkan 10 hingga 50 Gigabyte data log per hari. Di perusahaan berskala *Enterprise*, angka ini dengan mudah menembus hitungan Terabyte. Menemukan satu perintah *PowerShell* berbahaya di antara miliaran baris log sistem normal ibarat mencari jarum di tumpukan jerami.

Elastic Security—yang dibangun di atas fondasi Elasticsearch, Logstash, dan Kibana (ELK Stack)—adalah salah satu mesin pencari dan analitik keamanan paling kuat yang tersedia hari ini. Namun, antarmuka Kibana yang elegan sering kali menipu analis pemula. Banyak yang menggunakannya hanya seperti menggunakan mesin pencari web biasa: mengetikkan nama *file* `malware.exe` di bilah pencarian, dan berharap sistem memberikan jawaban.

Seorang *Threat Hunter* sejati tidak mencari jarum; mereka merancang magnet. Bab ini dirancang untuk membekali Anda dengan kemampuan menggunakan "magnet" tersebut. Kita akan membedah secara teknis bagaimana mengekstraksi nilai intelijen maksimum menggunakan **KQL (Kibana Query Language)** untuk penyaringan sekejap mata, dan **EQL (Event Query Language)** untuk melacak rangkaian jejak peretas lintas waktu.

1.2 Arsitektur ELK & Pipeline Data

Sebelum merancang kueri, *Threat Hunter* harus memahami topologi aliran data (*Data Pipeline*). Log tidak muncul secara magis di layar analis; mereka melewati pabrik pemrosesan yang kompleks.



Kesalahan terbesar analis junior adalah menulis kueri teks penuh (*Full-Text Search*) tanpa menentukan *Field*. Ini memaksa klaster Elasticsearch memindai miliaran dokumen secara keseluruhan (*Full Scan*), membebani CPU, dan memperlambat pencarian dari 1 detik menjadi 5 menit.

1.3 Standarisasi ECS (Elastic Common Schema)

Rahasia terbesar dari Elastic Security adalah ECS. Tanpa ECS, perburuan lintas-sistem adalah mimpi buruk. Jika Checkpoint Firewall menyebut IP penyerang `src`, dan CrowdStrike menyebutnya `LocalAddress`, Anda harus menulis dua kueri berbeda. ECS menormalisasi semuanya di dalam *Ingest Pipeline*.

BIDANG ECS UTAMA (KQL FIELD)	KONTEKS ANALITIK (TUJUAN TRIAGE)
`event.category`	Filter makro (`process`, `network`, `iam`). Wajib ada di setiap kueri perburuan.
`process.name`	Nama *file execution* (`cmd.exe`, `bash`). Sering dimanipulasi peretas.
`process.command_line`	Teks parameter penuh (`powershell -enc aB3...`). Area berburu paling kaya indikator.
`source.ip`	Arah asal serangan (IP Publik/Lokal). Telah dinormalisasi dari puluhan vendor.
`user.name`	Identitas akun yang terekam. Kunci untuk menemukan *Lateral Movement*.
`dns.question.name`	Domain yang dicari. Digunakan untuk mendeteksi C2 *Beaconing* atau Eksfiltrasi DNS.

Saat Anda berburu, biasakan selalu menggunakan skema ini: `[ecs.field] : "[value]"` untuk menjamin efisiensi basis data.

1.4 KQL vs EQL: Stateful vs Stateless

KQL (*Kibana Query Language*) adalah bahasa pencarian **Stateless**. Ia melihat setiap baris log sebagai entitas yang berdiri sendiri. Sangat hebat untuk menjawab: *"Apakah IP 8.8.8.8 ada di jaringan kita hari ini?"*

EQL (*Event Query Language*) adalah bahasa pencarian **Stateful / Temporal**. Ia mampu merangkai kejadian. Sangat hebat untuk menjawab: *"Apakah ada proses Word yang menelurkan PowerShell, yang kemudian menghubungi IP 8.8.8.8 dalam waktu 5 menit?"*

```
/* Contoh KQL: Cepat, tetapi sangat berisik (False Positive tinggi) */ process.name: ("cmd.exe" OR
"powershell.exe") AND network.direction: "egress" /* ------------------------------------------------
----- */ /* Contoh EQL: Membunuh noise dengan mengunci korelasi urutan waktu */ sequence by host.name,
user.name with maxspan=3m [process where process.name == "winword.exe"] [process where
process.parent.name == "winword.exe" and process.name == "powershell.exe"] [network where process.name
== "powershell.exe" and network.direction == "egress"]
```

EQL mengubah ratusan *alert* sampah menjadi 1 peringatan *Critical* yang 99% pasti merupakan infeksi *Macro Malware*.

BAB 2: ADVANCED THREAT DETECTION

2.1 Machine Learning di Lingkungan SOC

Bahkan EQL yang paling canggih pun memiliki batasan: ia membutuhkan manusia untuk mendefinisikan "apa yang jahat". Bagaimana jika peretas melakukan serangan lambat (*Low and Slow*) yang belum pernah ada polanya (*Zero-Day*)?

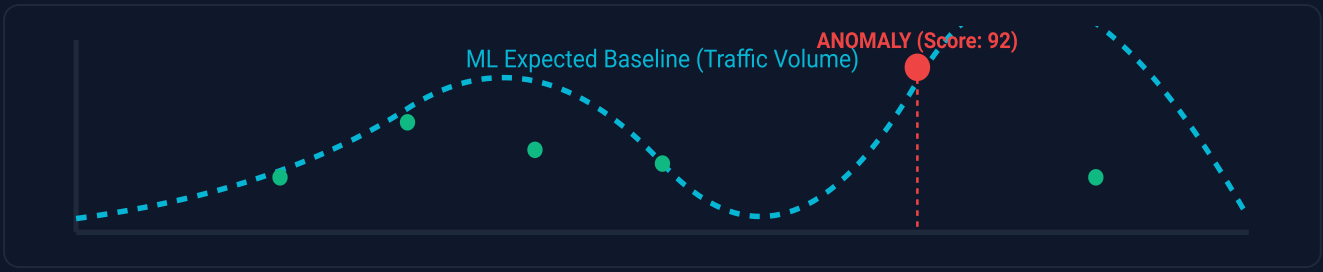
Di sinilah **Machine Learning (ML)** mengambil alih. Modul ML dalam SIEM modern (seperti Elastic ML atau Splunk UBA) beroperasi menggunakan algoritma *Unsupervised Learning* (Pembelajaran Tak Terawasi). Mesin tidak mencari tanda tangan (*signature*) virus. Ia mencari **Penyimpangan Basis (Baseline Deviation)**.

Secara operasional, modul ML membutuhkan waktu *training* pasif selama 7 hingga 14 hari di jaringan Anda. Selama periode ini, mesin merekam ritme harian setiap karyawan: jam berapa mereka *login*, aplikasi apa yang mereka buka, berapa *megabyte* data yang mereka kirim ke internet, dan ke negara mana data tersebut mengalir.

Setelah *baseline* terbentuk, setiap perilaku yang menyimpang dari kurva distribusi normal (*Gauss*) akan menghasilkan *Anomaly Score*. Jika skor ini menembus angka 75 dari 100, *alert* akan diteruskan ke antrean analis.

2.2 Membangun Anomaly Baseline

Visualisasi di bawah menggambarkan bagaimana mesin mendeteksi eksfiltrasi data (pencurian data) yang dilakukan pada dini hari, sesuatu yang pasti terlewatkan oleh ambang batas (*threshold*) KQL statis.



Pada pukul 2 pagi, meskipun volume data yang dikirim (misal 500MB) mungkin tidak melanggar aturan batas absolut *Firewall* (seperti batas 2GB), mesin menyadari bahwa *host* ini **secara historis tidak pernah** mentransfer data sebesar itu di luar jam kerja. Penyimpangan inilah yang memicu *alert* eksfiltrasi.

2.3 Tuning & Poisoning ML Jobs

Sistem *Machine Learning* tidak sempurna. Mereka rentan terhadap fenomena yang disebut **Data Poisoning** (Peracunan Data) yang terjadi secara tidak sengaja oleh operasi TI internal.

Jika tim *Network Engineering* memasang skrip *backup* baru yang berjalan setiap tengah malam dan mengirim 1 Terabyte data ke AWS, modul ML akan bereaksi keras selama hari ke-1 hingga hari ke-3 (menghasilkan *False Positive*). Namun, jika Anda tidak melakukan penyesuaian (*tuning*), pada hari ke-14, model ML akan "mempelajari" bahwa transfer 1 TB pada tengah malam adalah "normal". *Baseline* telah bergeser.

> **Praktik Engineering:**
> *Detection Engineer* harus mengecualikan (*exclude*) IP server *Backup* dan mesin *Vulnerability Scanner* dari input data *ML Job* SEBELUM *training* dimulai. Jika mesin belajar dari *noise*, ia akan menganggap serangan sebenarnya sebagai *noise*.

Dua tipe ML Job yang paling berharga untuk SOC: *Rare Process by User* (mendeteksi *Lateral Movement*) dan *Unusual DNS Question* (mendeteksi *C2 Beaconing/DGA*).

BAB 3: THE DETECTION BLUEPRINT

3.1 Mengelola Kebisingan (Noise Management)

Sebuah SIEM yang baru diinstal akan menghasilkan ribuan *alert* peringatan per jam jika Anda sekadar mengaktifkan "Aturan Standar" dari vendor. Sebagai Arsitek SOC, tugas Anda bukan mengaktifkan semuanya, melainkan membangun **Corong Deteksi (Detection Funnel)** yang hanya menjatuhkan peringatan valid ke meja analis L1.

Setiap aturan deteksi (*Rule*) memiliki empat kemungkinan hasil: *True Positive* (Serangan nyata ditangkap), *True Negative* (Aktivitas normal diabaikan), *False Positive* (Normal ditandai sebagai serangan), dan *False Negative* (Serangan lolos tak terdeteksi).

Mengurangi *False Positive* (FP) adalah prioritas harian. Jika *Precision Rate* (Tingkat Presisi) sebuah *Rule* berada di bawah 60% (artinya lebih dari 40% *alert* yang dihasilkannya adalah sampah), *Rule* tersebut lebih merusak moril analis daripada manfaat keamanannya, dan harus segera dimatikan untuk proses *Tuning*.

3.2 Whitelisting vs Suppression

Banyak analis junior menyamakan *Whitelisting* dengan *Suppression*. Keduanya merupakan teknik mengurangi *noise*, namun beroperasi di lapisan yang sepenuhnya berbeda.

PARAMETER	WHITELISTING (FILTER PENGECUALIAN)	SUPPRESSION (PEREDAMAN PERINGATAN)
Lokasi Logika	Di dalam Kueri KQL itu sendiri (contoh: AND `NOT source.ip: 10.x.x.x`).	Di lapisan *Alerting Engine* SIEM (pasca-kueri).
Penyimpanan Log	Aktivitas TIDAK dicatat sebagai ancaman di *Database Alert*. Hilang sepenuhnya.	Aktivitas TETAP dicatat di *Database* untuk forensik, hanya saja pop-up UI dibungkam.
Risiko Keamanan	**Tinggi.** Jika IP yang di-*whitelist* diretas, SIEM akan sepenuhnya buta (*Blind Spot*).	**Rendah.** Menggabungkan insiden berulang menjadi 1 tiket agar analis tidak kebanjiran (*spam*).

Aturan emasnya: Gunakan *Whitelisting* HANYA untuk pemindai kerentanan resmi perusahaan (seperti Nessus). Untuk anomali perilaku administratif berulang, selalu gunakan *Suppression Windows*.

3.3 Logika Threshold & Agregasi

Mengubah pencarian peristiwa tunggal (*Single Event*) menjadi ambang batas (*Threshold*) adalah cara tercepat membunuh *False Positive*. Misalnya, deteksi *Brute Force* tidak boleh menyala hanya karena 1 kegagalan kata sandi.

```
# Konfigurasi Threshold (Kondisi Pseudo-Sigma) detection: selection: event.category: "authentication"
event.outcome: "failure" network.direction: "inbound" # Logika Agregasi Kritis condition: selection |
count(user.name) by source.ip > 50 timeframe: 5m
```

Kode di atas menginstruksikan SIEM: *"Kumpulkan semua *login* gagal dari luar. Kelompokkan berdasarkan IP peretas. Hitung berapa banyak akun unik yang mereka coba. Jika lebih dari 50 akun dalam 5 menit, maka ini adalah serangan Password Spraying massal. Bunyikan Alarm!"*

3.4 Shadow Mode & Quality Assurance

Tidak ada *Rule* yang boleh langsung diterjunkan ke lingkungan produksi (diberikan kepada L1 SOC) tanpa pengujian. *Detection Engineer* yang profesional menggunakan **Shadow Mode** (Mode Bayangan).

Dalam *Shadow Mode*, logika KQL/EQL diaktifkan secara nyata di *server* SIEM, tetapi tindakan pengiriman peringatannya (seperti membunyikan tiket Jira atau mengirim email ke SOC) dimatikan total. *Rule* ini dibiarkan menyala di latar belakang selama 7 hari (termasuk melewati akhir pekan untuk melihat perilaku *cron jobs*).

Setelah 7 hari, *Engineer* membuka log hasil *Rule* tersebut. Jika *Rule* tersebut menghasilkan 50.000 *hit*, maka logika tersebut cacat dan harus diperbaiki (*Tuning*). Jika hanya menghasilkan 2 *hit* dan keduanya adalah pengujian *Red Team*, *Rule* tersebut lulus QA (*Quality Assurance*) dan siap dinaikkan statusnya menjadi *Active Production*.



BAGIAN II

Orkestrasi Taktis,
Perimeter, & Respons Cloud

BAB 4: ARSITEKTUR SOAR

4.1 Otomatisasi Respons (Security Orchestration)

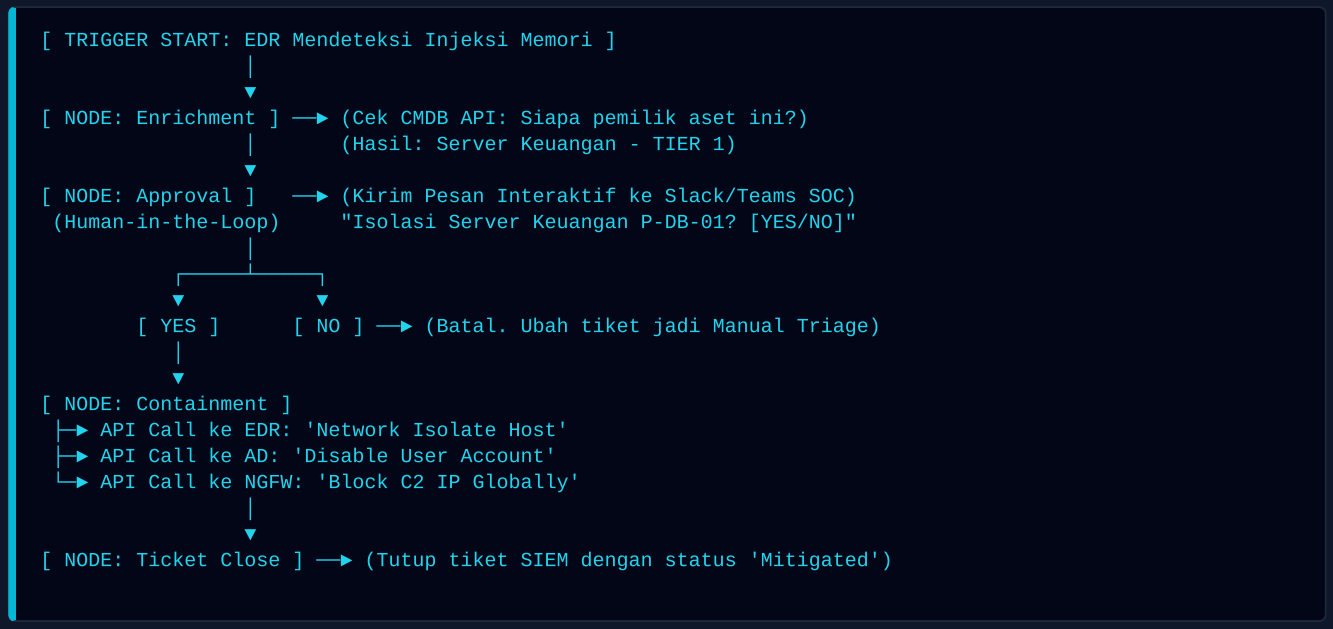
Jika SIEM bertindak sebagai mata dan telinga operasi keamanan, SOAR (*Security Orchestration, Automation, and Response*) adalah tangan dan kakinya. Tanpa SOAR, kecepatan respons SOC dibatasi oleh kecepatan analis mengetik *password* *firewall* dan mengeklik tombol *block* (*Human Speed*).

SOAR menghilangkan penundaan mekanis ini melalui **Playbooks**. *Playbook* adalah alur kerja visual (berupa kotak-kotak logika) yang membungkus puluhan panggilan API (*API Calls*) ke berbagai perangkat keamanan (EDR, Firewall, Active Directory) dalam hitungan milidetik setelah SIEM membunyikan alarm.

Arsitektur SOAR modern berfokus pada Pengayaan (*Enrichment*) dan Pengekangan (*Containment*). Mengotomatisasi penahanan *Ransomware* di jam 3 pagi saat analis manusia sedang kelelahan adalah metrik keberhasilan (ROI) tertinggi dari implementasi platform SOAR di tingkat *Enterprise*.

4.2 Malware Containment Playbook

Berikut adalah diagram alir (*flowchart*) taktis dari *Playbook* Karantina Otomatis untuk aset tingkat lanjut (*Tier 1*). Perhatikan penyisipan konsep *Human-in-the-Loop* (HitL) untuk mencegah pemadaman (*downtime*) sistem kritis secara tidak sengaja.



4.3 Integrasi API (Python ke EDR)

Di balik antarmuka visual *Playbook*, platform SOAR (seperti Cortex XSOAR atau Splunk Phantom) menjalankan skrip Python murni. Pemahaman rekayasa API (REST API) sangat krusial bagi arsitek SOC.

```
# Contoh potongan Python dalam Node SOAR untuk Isolasi Host (CrowdStrike) import requests def isolate_host(device_id, api_token): url = "https://api.crowdstrike.com/devices/entities/devices-actions/v2?action_name=contain" headers = { "Authorization": f"Bearer {api_token}", "Content-Type": "application/json" } payload = { "ids": [device_id] } response = requests.post(url, json=payload, headers=headers) if response.status_code == 202: return "Success: Host Network Isolated" return "Failed to isolate host"
```

Kode di atas mencabut antarmuka kartu jaringan (NIC) *host* yang terinfeksi dari jaringan korporat dalam hitungan milidetik, hanya menyisakan saluran komunikasi tunggal antara *host* tersebut dan konsol EDR (CrowdStrike) untuk forensik jarak jauh.

BAB 5: PERIMETER DEFENSE

5.1 Mempertahankan Garis Depan

Meskipun adopsi *Cloud* meningkat pesat, perimeter fisik pusat data (*Data Center*) masih merupakan arena pertempuran utama bagi perusahaan tingkat *Enterprise*. Pertahanan perimeter bukan sekadar "Blokir semua yang tidak dikenal". Ini adalah tentang inspeksi cerdas terhadap paket jaringan menggunakan *Next-Generation Firewall* (NGFW) seperti Palo Alto, Checkpoint, atau Fortinet.

NGFW menembus keterbatasan *firewall* tradisional (yang hanya melihat IP dan Port) dengan melakukan *Deep Packet Inspection* (DPI). Ia mampu menerjemahkan lalu lintas di port 443 menjadi nama aplikasi spesifik (misal: "Facebook Chat" vs "Facebook Login") dan mengikat lalu lintas tersebut pada identitas pengguna (*Identity Awareness*).

Bagi analis SOC, tantangan terbesar dari NGFW adalah mengelola volume log yang sangat masif. Mengirim 100% log NGFW ke SIEM akan menghabiskan kuota lisensi Anda dalam sehari. Kebijakan penyaringan log (*Log Filtering Policy*) yang ketat harus diterapkan di *pipeline* (seperti *Logstash*) sebelum data menyentuh *database* SIEM.

5.2 Ingress vs Egress Monitoring

Lalu lintas perimeter dibagi dua: *Ingress* (Masuk dari internet ke dalam) dan *Egress* (Keluar dari dalam ke internet). SOC harus memprioritaskan energi mereka pada Egress.

PARAMETER	INGRESS (LALU LINTAS MASUK)	EGRESS (LALU LINTAS KELUAR)
Sifat Data	99% *Noise* (Pemindaian botnet acak dari internet).	Campuran antara *Browsing* karyawan yang sah dan Eksfiltrasi Malware.
Fokus Analis	Abaikan blokir (*Drop*). Awasi dengan ketat *Login* VPN dan Serangan Aplikasi Web (WAF).	Fokus Maksimal. Deteksi *Command & Control* (C2), DNS Tunneling, dan unggahan data masif.
Prioritas Rule	Rendah (Rawan *False Positive* akibat *Scanner Hits*).	Sangat Tinggi (Titik cekik utama eksfiltrasi peretas).

Aturan (*Rule*) Egress yang paling berharga: Pantau peladen (*Server*) internal yang mencoba berkomunikasi langsung ke IP publik mentah tanpa resolusi nama *Domain* (FQDN), sebuah indikator klasik dari *malware* modular.

5.3 External Dynamic Lists (EDL)

Jika analis SOC menemukan 1.000 IP botnet yang harus diblokir malam ini, mereka tidak akan membuka konsol *firewall* dan menempelkannya satu per satu. Mereka menggunakan **External Dynamic Lists (EDL)**.

```
# Contoh File blocklist.txt (Di-host di server SOC internal) # NGFW mengambil (fetch) file ini setiap 5 menit via HTTP 185.10.x.x 89.44.x.x 45.33.x.x # SOAR Script Python menambahkan IP ke file ini secara otomatis
```

Dengan mengonfigurasi *Rule* blokir pada NGFW agar merujuk (*point*) ke URL EDL ini (contoh: `http://soc-internal.net/blocklist.txt`), *firewall* akan membarui memori aturannya secara dinamis. Metode ini menjamin *Zero-Downtime Containment* karena tidak mengharuskan admin jaringan melakukan *Push/Install Policy* yang berisiko mengganggu koneksi pengguna yang sedang aktif (*Stateful Connections*).

BAB 6: CLOUD INCIDENT RESPONSE

6.1 Respons Insiden di Lingkungan Awan

Respons Insiden (IR) di lingkungan *Cloud* (AWS/Azure) membuang konsep isolasi kabel jaringan atau pencabutan *hard disk*. Anda tidak bisa mencabut kabel LAN dari *server* Amazon.

Di *Cloud*, aset bersifat *ephemeral* (sesaat)—dibuat dan dihancurkan oleh kode (*Terraform/K8s*) dalam hitungan menit. Pendekatan perburuan ancaman bergeser dari penganalisis paket PCAP menuju analisis log **Control Plane** (Lapisan Kontrol Manajemen).

Jika peretas menyusup ke lingkungan AWS Anda, hadiah utama mereka bukanlah data di dalam *Virtual Machine* (EC2), melainkan **Kredensial IAM (Identity and Access Management)**. Dengan IAM yang cukup kuat, peretas dapat menyandera seluruh pusat data *Cloud* Anda, menghapus *backup* (*S3 Buckets*), dan menambang *Crypto* menggunakan seratus *server* terbesar atas tagihan kartu kredit perusahaan Anda.

6.2 Analisis Log CloudTrail

Di ekosistem AWS, *CloudTrail* adalah mata dewa (*God's Eye*). Log ini mencatat setiap panggilan API yang dilakukan oleh pengguna, layanan, atau peretas ke infrastruktur Anda. *Detection Engineer* harus menulis kueri KQL khusus untuk menangkap anomali IAM di *CloudTrail*.

```
/* KQL: Deteksi API CloudTrail yang sangat berbahaya (Recon & Escalate) */ event.module: "aws" AND
event.dataset: "aws.cloudtrail" AND aws.cloudtrail.event_name: ( "AttachUserPolicy" OR "CreateAccessKey"
OR "AssumeRole" ) AND NOT source.ip: "*IP_VPN_Perusahaan*"
```

Logika di atas mencari aktivitas "Eskalasi Hak Istimewa" yang sangat jarang dilakukan oleh pengguna normal, dan memastikan *alert* berbunyi Keras jika perintah ini dieksekusi dari Alamat IP di luar jaringan VPN aman milik kantor.

6.3 IAM Quarantine Automation

Karantina di *Cloud* tidak berpusat pada pemblokiran IP, karena peretas dapat menggunakan proksi atau VPN publik yang terus berganti IP. Titik cekik karantina *Cloud* adalah **Identitas (IAM)**.

> **Taktik Isolasi Cloud (AWS Deny All):**
> Saat SOC mengonfirmasi sebuah kredensial AWS telah bocor ke *GitHub*, platform SOAR langsung mengeksekusi API `PutUserPolicy`.
>
> Kebijakan ini menyisipkan dokumen JSON: `{"Effect": "Deny", "Action": "*", "Resource": "*"}` secara paksa ke identitas (*Role/User*) yang dikompromikan.
>
> Ini secara instan memutuskan seluruh sesi terminal dan akses API musuh di seluruh dunia, mengunci mereka di luar ruang konsol (*Console*) tanpa harus mematikan (*shutdown*) server aplikasi klien (EC2) yang mungkin terkait.

Kecepatan eksekusi *Playbook IAM Quarantine* ini (harus di bawah 3 menit sejak bocor) menentukan kelangsungan hidup data perusahaan di awan.



BAGIAN III

Proaktif, Emulasi Ancaman,
& SecOps Tooling

BAB 7: PURPLE TEAMING

7.1 Metodologi Emulasi Ancaman

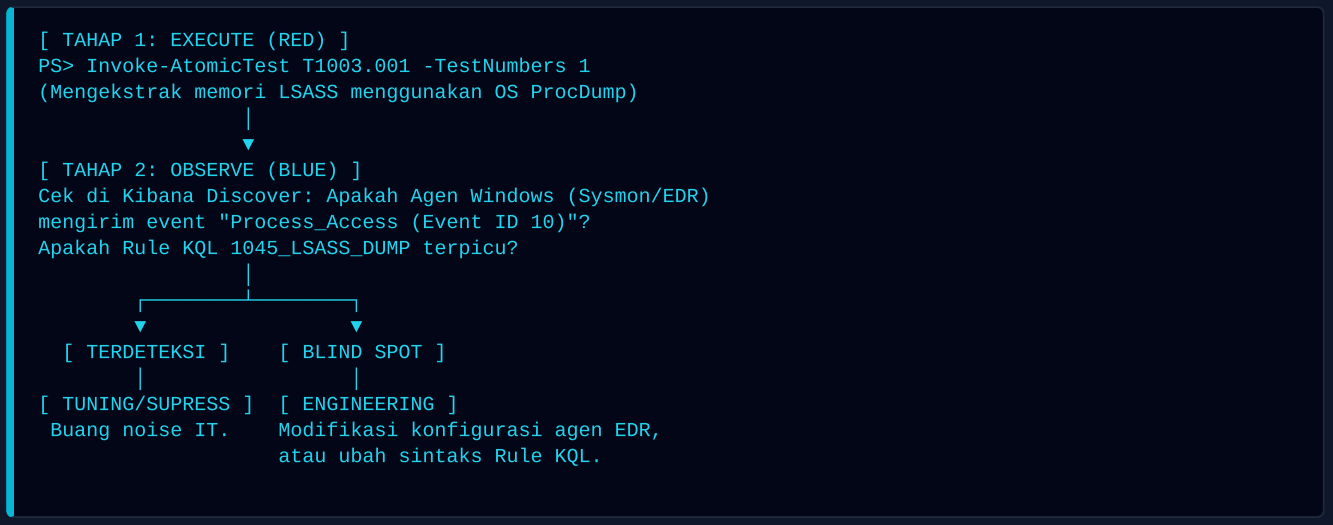
Sebuah aturan deteksi (*Rule*) di SIEM hanyalah sebuah hipotesis sampai ia diuji oleh serangan nyata. Secara historis, tim *Red* (Offensif) dan *Blue* (Defensif) bekerja dalam silo yang bermusuhan. **Purple Teaming** adalah metodologi fusi; ini bukan nama sebuah tim baru, melainkan cara kerja (*framework*).

Dalam *Purple Teaming*, serangan diemulasikan secara terukur (*Controlled Execution*) oleh alat otomatis, sementara *Detection Engineer* mengamati log EDR dan SIEM pada detik yang sama untuk memvalidasi apakah telemetri ditangkap dengan benar, dan apakah *alert* dipicu (*fired*).

Kita tidak mencari kerentanan perangkat lunak (*software vulnerability / CVEs*). Kita mencari kerentanan **Deteksi (Detection Gaps)**. Jika *malware* mengubah teknik pencurian *password*-nya, apakah KQL kita masih cukup tangguh untuk mendeteksinya berdasarkan *Perilaku*, bukan sekadar namanya?

7.2 Atomic Red Team Emulation Flow

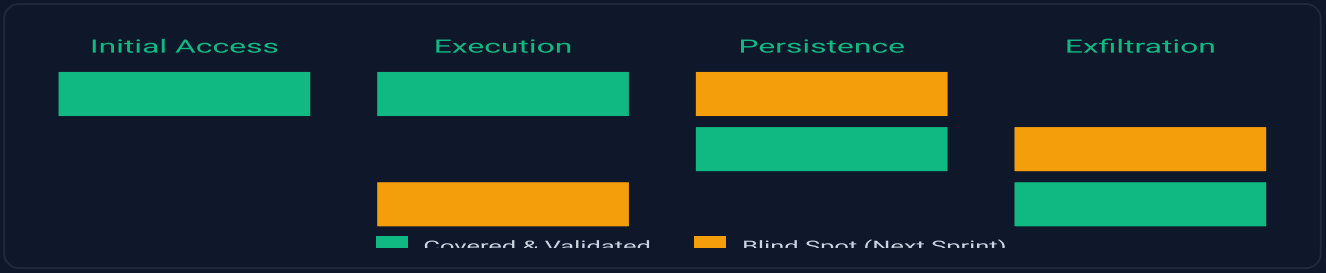
Kerangka kerja (*Framework*) paling populer untuk validasi ini adalah **Atomic Red Team** dari Red Canary. Ia menyediakan ratusan skrip serangan kecil (*Atomic Tests*) yang terpetakan langsung ke matriks MITRE ATT&CK.



Uji coba berulang (*Iterative Retesting*) inilah yang mengasah pertahanan SOC dari pisau tumpul menjadi pedang bedah.

7.3 Visualisasi MITRE Heatmap

Untuk melaporkan kesiapsiagaan (*Readiness Posture*) kepada CISO, *Detection Engineer* memvisualisasikan "Cakupan Deteksi yang Tervalidasi" dalam bentuk Peta Panas (*Heatmap*) matriks MITRE ATT&CK.



Kotak kuning (*Blind Spot*) menentukan prioritas sprint rekayasa (*Engineering Sprint*) di minggu berikutnya. Jangan mengejar 100% hijau, karena beberapa taktik MITRE merupakan perilaku Admin sistem yang wajar (terlalu banyak *noise*).

BAB 8: SECOPS TOOLING

8.1 Custom Triage Interfaces

Platform SIEM dan SOAR tingkat *Enterprise* sangat kuat, namun sering kali UI mereka kaku dan lambat untuk tugas triase cepat. Analis sering kali membuang waktu 10 menit hanya untuk menyalin alamat IP dari log teks mentah dan memindahkannya ke banyak *tab browser* intelijen ancaman (*Threat Intel*).

Insinyur SOC yang mahir tidak menunggu vendor memperbaiki UI; mereka merakit alat (*tool*) mereka sendiri. **Standalone Tactical Tools** adalah aplikasi web ringan berbasis peramban (*Single Page Applications*) yang dirancang dengan HTML dan Tailwind CSS.

Alat ini bertindak sebagai *IoC Extractor* lokal. Anda cukup menempelkan (*paste*) ribuan baris *Header Email* atau log Windows, dan dalam hitungan milidetik, alat tersebut menyajikan lencana indikator (*IP, Hash, URL*) yang sudah difilter dari duplikat dan dihipertautkan (*hyperlinked*) langsung ke *VirusTotal*.

8.2 JavaScript RegEx Parser (Client-Side)

Penguasaan *Regular Expressions* (RegEx) adalah kunci otomasi *parsing*. Kode di bawah ini adalah otak (*Engine*) dari alat ekstraktor Anda yang beroperasi murni di RAM *browser* lokal (sangat aman untuk data rahasia/log perusahaan).

```
// Fungsi ekstraksi kilat dari tumpukan Raw Logs di Textarea UI function extractIoCs(rawText) { // Pola RegEx untuk mendeteksi IPv4 dan MD5 Hash const regexIP = /\b(?:[0-9]{1,3}\.){3}[0-9]{1,3}\b/g; const regexMD5 = /\b[A-Fa-f0-9]{32}\b/g; // Array.match() menarik semua indikator. Jika Null (tak ada), kembalikan Array kosong [] const rawIPs = rawText.match(regexIP) || []; const rawMD5s = rawText.match(regexMD5) || []; // Ekstrak & hilangkan duplikasi seketika (Gunakan struktur data JS Set) // Ini mengubah 5000 IP botnet yang sama menjadi 1 entitas IP bersih. const uniqueIps = [...new Set(rawIPs)]; const uniqueMd5s = [...new Set(rawMD5s)]; return { ips: uniqueIps, md5s: uniqueMd5s }; }
```

Dengan teknik penghapusan duplikasi (*Deduplication*) via `Set`, alat ini memangkas beban kognitif (*cognitive overload*) pada antarmuka (UI) analis secara instan.

8.3 Fetching API & Mengatasi CORS

Alat triase menjadi senjata mematikan ketika ia tidak hanya mengekstrak IP, tetapi juga menarik skor ancaman secara *real-time* (Enrichment). Ini dilakukan via API `fetch()` bawaan *Browser*.

```
// Fungsi Asinkron untuk Fetch Geolokasi dari AbuseIPDB / IP-API async function enrichIP(ipAddress) {
try { // Gunakan API yang mendukung panggilan langsung (CORS-friendly) const response = await
fetch(`http://ip-api.com/json/${ipAddress}?fields=country,isp`); const data = await response.json();
return `${ipAddress} [${data.country}]`; // Contoh: "8.8.8.8 [US]" } catch (error) { return
`${ipAddress} [API Limit/Error]`; } }
```

OpSec Peringatan:
Jangan pernah *Hardcode* API Key (seperti kunci VirusTotal) ke dalam *script* .html Anda! Siapa pun yang membuka alat tersebut dan menekan Inspect Element (F12) dapat mencuri kuota perusahaan. Buatlah kolom *Input API Key* di UI agar analis memasukkannya di sesi RAM sementara (*sessionStorage*).

BAB 9: INTERACTIVE SECOPS DASHBOARDS

9.1 Estetika yang Menyelamatkan Nyawa

Layar komando (*Dashboards*) di ruang operasi sering kali dirancang oleh desainer metrik yang tidak pernah merasakan tekanan bekerja di bawah status *Incident Response*. Akibatnya, layar dipenuhi oleh bagan kue (*pie charts*) 3D berputar dan peta bumi interaktif yang memakan banyak ruang tetapi memberikan nol (0) nilai intelijen aksi (*actionable intel*).

UI Dasbor Keamanan (*Security Dashboard*) yang brilian berfokus pada **Data-to-Ink Ratio**—membuang setiap piksel warna, bayangan (*drop-shadow*), dan animasi yang tidak memberikan informasi keamanan secara langsung.

Target dasbor utama SOC (*Triage Board*) harus bisa menjawab 3 pertanyaan ini dalam waktu kurang dari 5 detik lirikan mata:
1. Berapa banyak peringatan yang belum diklaim (*Unassigned*)?
2. Apa Level Bahaya Tertinggi (*Highest Severity*) saat ini?
3. Aset penting apa yang saat ini sedang diserang?

9.2 Arsitektur Bento Grid Layout

Untuk menghindari pemuatan halaman ganda (*page reload*) atau navigasi yang dalam (*horizontal scrolling*), dasbor SOC modern mengadopsi arsitektur *Bento Grid*—memecah aplikasi ke dalam kotak-kotak (*Cards*) informasi yang padat dalam *Single Page Application*.

- **Top KPI Banner (Strip Metrik):** Pita sempit di posisi paling atas. Menampilkan angka-angka krusial: *Mean Time to Detect (MTTD)*, Total Open Cases, dan *DEFCON Status*.
- **Main Stage (Feed Tabel):** Menguasai 60% porsi layar (kiri ke tengah). Berisi daftar antrean *Alert* yang menggulir (*scroll*) secara virtual. Menggunakan tipografi *Monospace* (`10.0.0.5`) agar alamat IP dan jam (*timestamp*) sejajar tegak lurus.
- **Context Drawer (Laci Kanan):** Menguasai 40% sisi kanan. Bertindak sebagai *Inspector Panel*. Ketika analis mengklik salah satu baris di *Feed Tabel*, laci kanan ini langsung terisi dengan data skor *Threat Intel*, avatar aset target, dan tombol merah "Isolate Host". Nol *Swivel-Chair Time* (Tidak perlu ganti *tab browser*).

9.3 Psikologi Warna & Ergonomi Kognitif

Mode Gelap (*Dark Mode* dengan basis `Slate 900`) bukan soal gaya *hacker* (estetika semata), melainkan Kesehatan dan Keselamatan Kerja (K3) untuk melindungi retina analis dari layar yang menyilaukan selama rotasi *shift* 12 jam.

TINGKAT (SEVERITY)	KODE HEX / CSS CLASS	ATURAN PENERAPAN (RULES OF ENGAGEMENT)
CRITICAL	`#ef4444` (Crimson Red)	Hanya digunakan jika eksploitasi BERHASIL (misal: *Ransomware* bergerak). Terlalu banyak merah akan merusak sensitivitas kepanikan analis (*Alert Fatigue*).
HIGH	`#f59e0b` (Amber / Orange)	Serangan berpotensi fatal yang berhasil digagalkan EDR, atau *Password Spraying* masif.
MEDIUM	`#0ea5e9` (Cyber Blue)	Anomali IAM, Perubahan konfigurasi aneh. Warna biru ini menarik mata namun menenangkan (tidak mendikte kepanikan).
LOW/INFO	`#64748b` (Muted Slate)	Pemindaian *Port* acak dari luar (*Noise*). Dibuat seteduh mungkin agar memudar ke latar belakang.



BAGIAN IV

Kecerdasan Buatan (AI),
LLMs, & Otomatisasi Lanjutan

BAB 10: AI-DRIVEN SOC (LLMS & RAG)

10.1 Menggeser Beban Analis ke AI

Puncak dari rekayasa arsitektur SOC modern adalah pelimpahan investigasi tingkat rendah (*Low-Level Triage*) ke agen kecerdasan buatan mandiri. *Large Language Models* (LLMs) memiliki kemampuan superior untuk membaca bongkahan (*chunks*) JSON dari sistem EDR dan menyimpulkan rangkaian niat serangannya.

Namun, dalam industri korporat dan perbankan, mengunggah data (*copy-paste*) log rahasia ke ChatGPT publik merupakan pelanggaran kepatuhan (*Compliance Violation*) tingkat tinggi. Model AI Keamanan harus diputar secara *On-Premise* (Lokal) menggunakan model terbuka (*Open Weights*) seperti Llama 3 atau Mistral yang berjalan di *server* GPU *air-gapped*.

Masalah berikutnya: LLM dasar (*Base Model*) tidak mengetahui letak *server* penting Anda atau kebijakan respons insiden internal perusahaan Anda. LLM tersebut sering kali berhalusinasi. Solusinya? Kita harus mencangkokkan "memori khusus perusahaan" ke dalam AI melalui arsitektur **Retrieval-Augmented Generation (RAG)**.

10.2 Arsitektur RAG (Retrieval-Augmented Gen)

RAG bekerja layaknya buku panduan terbuka (*Open-Book Exam*) bagi AI. Sebelum AI menjawab, sistem menarik dokumen *Playbook Internal* SOC Anda dan membekalkannya kepada AI.

```
[ 1. TRIGGER ] ──► (Alert Baru: SQL Injection di Server B)
      │
      ▼
[ 2. RETRIEVAL (Vector DB) ]
Sistem mencari di Database Vektor internal SOC:
"Apa dokumen SOP penanganan SQLi pada Server B?"
(Mendapatkan: Dokumen SOP v2.4 dan Topologi Jaringan).
      │
      ▼
[ 3. AUGMENTATION (Prompting) ]
Sistem menggabungkan RAW SIEM LOG + DOKUMEN SOP ke dalam
Satu Prompt Instruksi raksasa (System Context).
      │
      ▼
[ 4. GENERATION (Local LLM) ]
AI membaca prompt gabungan tersebut dan menghasilkan Output:
{"status": "False Positive", "reason": "Scanner Qualys Internal"}
```

Arsitektur RAG memastikan bahwa LLM hanya mengambil keputusan berdasarkan doktrin (*Playbook*) tertulis milik organisasi Anda, menjamin determinasi teknis tanpa risiko halusinasi.

10.3 System Prompt Engineering untuk JSON

Agar platform SOAR dapat memproses (*parse*) rekomendasi dari AI, LLM tidak boleh menghasilkan esai paragraf panjang. Anda harus melakukan *Prompt Engineering* paksa (*Strict Forcing*) agar agen AI menjawab murni dalam struktur objek JSON.

```
SYSTEM: You are an Expert L3 SOC Analyst System. Your task is to analyze the following raw JSON log and determine if it represents a True Positive. Use the attached Playbook Context to make your decision. PLAYBOOK CONTEXT: {rag_retrieved_internal_wiki_docs} RAW SIEM LOG: {elastic_raw_json} OUTPUT CONSTRAINT: Return ONLY a valid JSON object. Do not include markdown blocks, greetings, or conversational filler. EXPECTED SCHEMA: { "classification": "FP / TP", "confidence_score_percent": int, "mitre_technique_id": "TXXXX", "reason": "short technical explanation" }
```

10.4 LoRA Fine-Tuning pada LLM Lokal

Arsitektur RAG sangat brilian, namun terkadang AI masih berbicara dengan gaya yang terlalu santai atau menggunakan istilah keamanan (*terminologies*) yang salah.

Langkah akhir untuk mencapai AI yang sempurna adalah **LoRA (Low-Rank Adaptation) Fine-Tuning**. Ini adalah proses "melatih ulang" sebagian neuron model LLM secara efisien tanpa memerlukan superkomputer. Insinyur *Machine Learning* akan mengekstrak 5.000 tiket / Laporan Kasus penanganan insiden historis milik SOC (yang pernah ditutup oleh analis L3 Senior Anda di masa lalu) yang telah disanitasi.

Melalui proses *Fine-Tuning* ini, model LLM akan secara organik menyerap gaya penulisan, dialek teknis, dan insting deduksi khas analis senior perusahaan Anda. AI tersebut tidak lagi hanya menjadi asisten umum (*general assistant*), melainkan telah berevolusi menjadi replika virtual dari *Threat Hunter* paling berpengalaman di ruangan Anda.



BAGIAN V

Studi Kasus Forensik &
Penerapan Investigasi di Dunia Nyata

BAB 11: CASE STUDY - RANSOMWARE

11.1 Deteksi & Containment Cepat

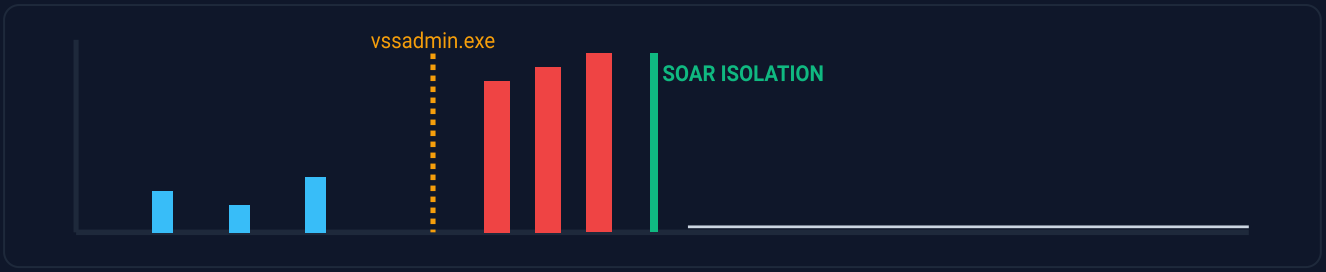
Tidak ada simulasi yang sebanding dengan krisis sebenarnya. Studi kasus pertama kita membedah serangan *Conti Ransomware* pada titik akhir (*endpoint*) Windows perusahaan. Bagaimana *Detection Engineering Blueprint* (Bagian I) berpadu dengan SOAR (Bagian II) menyelamatkan jutaan dolar dalam 4 menit.

Kronologi Triage:

- **T=0s:** *Payload* (muatan) lolos dari blokade NGAV tradisional karena menggunakan *Zero-Day Crypter* (dikemas/di-obfuskasi).
- **T=15s:** Peretas menggunakan LOLBin (`vssadmin.exe`) untuk menghapus cadangan *Shadow Copy*. EDR mengirim log *Process Creation* (Event ID 4688) ke *Pipeline* SIEM Elastic.
- **T=20s:** Aturan *Stateful* EQL di SIEM berteriak. Aturan (Rule) `Ransomware_Precursor_ShadowCopy_Delete` menghasilkan *Alert* tingkat *Critical*.
- **T=25s:** SOAR menarik *Alert* via REST API. *Playbook HitL* (Bab 4) dimulai secara instan. Menyadari target adalah *File Server* Tier 1, SOAR membunyikan PagerDuty analis L2.
- **T=90s:** Analis menyetujui (*Approve*) aksi isolasi via tombol di *Chat* Slack.
- **T=95s:** Server diisolasi (*Network Containment*). Kriptografi *Ransomware* dihentikan di tingkat *Memory* sebelum mengeksekusi iterasi ke-2 di *folder root*.

11.2 Attack Timeline Reconstruction

Setelah isolasi, tugas *Hunter* adalah rekonstruksi. Melalui fitur Kibana Lens, kita memvisualisasikan ritme *File Write* IOPS untuk mengonfirmasi titik akhir (*endpoint*) kerusakan.



Visualisasi garis waktu (*Timeline*) grafis membuktikan dengan meyakinkan kepada jajaran eksekutif bahwa otomatisasi SOC memotong lonjakan eksponensial fase enkripsi massal sebelum kerusakan merambat ke *Drive* sekunder, menyelamatkan infrastruktur inti.

BAB 12: CASE STUDY - INSIDER THREAT

12.1 Eksfiltrasi Data oleh Pihak Internal

Ancaman dari dalam (*Insider Threat*) lebih sulit dideteksi daripada peretas eksternal, karena pelaku telah memiliki kredensial resmi (*True Negative* dalam konteks autentikasi) dan menggunakan aksesnya yang valid untuk mencuri data.

Dalam skenario ini, seorang eksekutif akun yang akan mengundurkan diri (*resign*) mencoba menyalin ribuan berkas data nasabah (PDF & Excel) dari server Sharepoint internal dan memindahkannya ke layanan penyimpanan *Cloud* pribadi (Google Drive / Dropbox) via *browser* Chrome pribadinya.

Ambang batas *Firewall Egress* konvensional (KQL) sering mengabaikan lalu lintas 443 (HTTPS) ke Google Drive karena Google Drive di-*whitelist* sebagai layanan "halal" (*False Negative*). Ini adalah ranah perburuan yang mewajibkan kolaborasi antara Modul **Anomaly Machine Learning (ML)** dan **EQL Stateful Tracking**.

12.2 EQL Correlation Query Breakdown

Taktik kita adalah menyatukan dua indeks (*Endpoint File Events* dan *Network Flow*) menjadi satu cerita korelasi temporal yang mengerucut pada 1 identitas *User*.

```
/* EQL: Mass File Read followed by High Bandwidth Egress */ sequence by user.name with maxspan=15m /* 1. Threshold: Pengguna membaca >50 file sensitif dengan cepat */ [any where event.category == "file" and event.action == "read" and file.path : "*\\SharePoint\\Confidential\\*"] /* 2. Disusul oleh transfer data masif ke luar (Egress) */ [network where network.direction == "egress" and network.bytes_written > 100000000 /* >100MB */ and not destination.domain : "*.perusahaan.internal"]
```

Meskipun Google Drive (`destination.domain`) sah (*legit*), kombinasi dari *Membaca massal folder rahasia* DIIKUTI oleh *Transfer Egress >100MB* dalam jeda waktu 15 menit menciptakan pola niat (*Intent Pattern*) tingkat tinggi yang tidak mungkin dilakukan secara kebetulan oleh operasional biasa.

GLOSARIUM SECOPS (A-M)

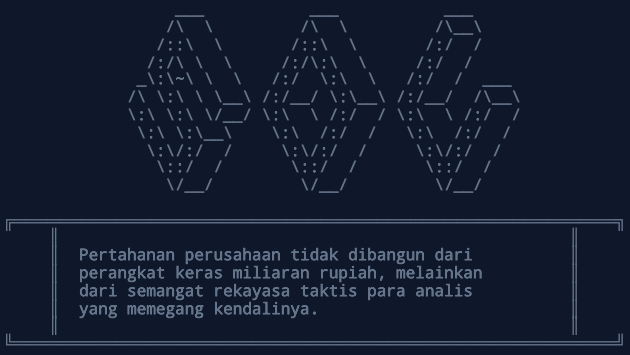
Kamus operasional bagi *Detection Engineers* & SOC Analysts.

TERMINOLOGI	DEFINISI KONTEKSTUAL & OPERASIONAL
Alert Fatigue	Kelelahan kognitif analis akibat pembombardiran terus-menerus ribuan *False Positive* dari SIEM.
Baseline Deviation	Pendekatan Machine Learning (ML) yang menandai aktivitas normal (*baseline*), lalu membunyikan peringatan saat ada penyimpangan (anomali) ekstrem.
Chokepoint Tactics	Membangun aturan (*Rules*) dengan presisi ketat pada fase serangan paling sempit/wajib dilewati oleh peretas (misal: *Privilege Escalation*).
CORS	Mekanisme keamanan peramban *web* yang mencegah pembacaan data API dari domain yang berbeda; tantangan utama dalam *SecOps Tooling* berbasis *Client-Side*.
ECS	*Elastic Common Schema*. Standar normalisasi agar data dari 100 *vendor* EDR/Firewall memiliki nama *field* yang seragam di SIEM.
EQL	*Event Query Language*. Mesin pencari korelasional (*Stateful*) untuk melacak urutan (*sequence*) serangan secara kronologis.
HitL (Human in the Loop)	Konsep desain SOAR di mana eksekusi akhir (misal: mematikan *Server Core*) dihentikan sementara (*pause*) untuk menunggu 1 klik persetujuan (*Approval*) dari analis manusia (mencegah otomatisasi yang merusak).
LOLBins	*Living Off the Land Binaries*. Alat sistem asli bawaan OS (misal: PowerShell, WMI) yang disalahgunakan peretas agar tidak diblokir Antivirus.

GLOSARIUM SECOPS (N-Z)

TERMINOLOGI	DEFINISI KONTEKSTUAL & OPERASIONAL
Precision vs Recall	Presisi: "Berapa banyak *alert* yang benar-benar serangan?" (Kualitas). Recall: "Berapa banyak serangan asli yang gagal kita deteksi?" (Sensitivitas).
Purple Teaming	Fusi *Red Team* dan *Blue Team*. Mensimulasikan satu taktik peretas dan memastikan kueri *Blue Team* menangkap *log* tersebut di detik yang sama.
RAG	*Retrieval-Augmented Generation*. Arsitektur AI di mana *Prompt* LLM disuntikkan dengan *Playbook* perusahaan agar AI tidak berhalusinasi saat menjawab triase insiden.
Shadow Mode	Uji coba (*QA*) aturan KQL baru di *Production* SIEM selama 7 hari tanpa membunyikan notifikasi *Alert*, murni untuk memantau potensi rasio *False Positive*.
Sigma Rules	Bahasa *open-source* (format YAML) penulisan deteksi ancaman (*Detection as Code*). Mampu diterjemahkan secara fleksibel ke Splunk SPL atau Elastic KQL.
Suppression Window	Filter SIEM tingkat peringatan (*Alert Level*). Jika 1 IP menembak 5.000 kali dalam 1 jam, sistem "meredam" (*Suppress*) 4.999 *alert* agar analis hanya mendapat 1 tiket kumulatif.
Whitelisting	Filter KQL (*Rule Level*) absolut. Aktivitas yang di-*whitelist* akan diabaikan total oleh SIEM (sangat berbahaya jika digunakan sembarangan, bedakan dengan *Suppression*).

Glosarium ini mengakhiri pedoman taktis SecOps. Terminologi ini bukan sekadar definisi, melainkan pilar-pilar operasional yang membedakan seorang pembaca log dengan seorang Arsitek Keamanan.



Pertahanan perusahaan tidak dibangun dari perangkat keras miliaran rupiah, melainkan dari semangat rekayasa taktis para analis yang memegang kendalinya.

Berdiri di Garis Depan (Stand the Watch)

© 2026 Rizki Angga Pratama.
Semua Hak Cipta Dilindungi.
Edisi Volume I